

1 Descripción

En este documento se presenta un método para determinar la expresión analítica de los ceros de una función de $\mathbb{R}^n \rightarrow \mathbb{R}^n$.

Lo primero es importar las bibliotecas que vamos a usar.

2 importar bibliotecas

```
1 from scipy.optimize import root
2 import pandas as pd
3 import numpy as np
4 from random import uniform, randint
```

3 definir la función que nos interesa

El próximo paso es definir una función con la que probar el algoritmo. Por ahora, solo para que sea sencillita vamos a considerar la función:

$$F(x) = \begin{pmatrix} (x-a)(y-ab) \\ xy-ab^2 \end{pmatrix}^t$$

Esta función tiene la característica que sus ceros son de la forma: (b, ab) . Sin embargo, la primera componente tiene un “cero frontera” en $x = a$. Veamos cómo el sistema logra lidiar con esto.

Lo primero es definir la función.

```
1 def function1 (x,p):
2     x1 = x[0]
3     x2 = x[1]
4     a = p[0]
5     b = p[1]
6     f = [(x1-a)*(x2-a*b), x1*x2-a*b*b]
7     return f
```

Vamos a comprobar que la función evalúa como debe:

```

1 print(function1([1,2],[3,4]))
2 print("El siguiente debe ser (0,0)")
3 print(function1([4,8], [2,4]))
4 print(function1([2,8], [2,4]))

```

Resultado:

```

1 [20, -46]
2 El siguiente debe ser (0,0)
3 [0, 0]
4 [0, -16]

```

Listo, ya tenemos importadas todas las bibliotecas que nos interesan y tenemos definida la función. Vamos a generar los datos para este problema.

4 generar datos y guardarlos en un csv

La “generación de los datos” consiste en lo siguiente.

Se tienen cotas para los posibles valores de los parámetros, y cotas para los posibles valores de los puntos iniciales.

A partir de esas cotas se generan valores para los parámetros y para cada combinación de los parámetros se generan varios puntos iniciales para buscar mínimos.

Los datos deberían ser de la forma:

p1	x01 ₁
p1	x01 ₂
p1	x01 ₃
p2	x02 ₁
p2	x02 ₂
p2	x02 ₃
...	...

Esto sería en el caso en el que generamos 3 valores iniciales por cada valor del parámetro.

Ahora generamos la primera parte los datos para la evaluación

Aquí sería interesante saber qué significa cada parámetro.

Parece que

L es la cantidad de repeticiones que se van a hacer,

lx0 la fila en la que guarda los valores iniciales

lp la fila en la que se guarda los parámetros generados

pero creo que esto está mal :-(...

La idea era generar varios valores para los parámetros.

Después, **por cada uno de esos valores**, generar varios puntos iniciales

Así que en realidad,

Vamos a cambiar esos nombres de variables y definir la cantidad de parámetros que vamos a generar y la cantidad de puntos iniciales por parámetros.

```
1 cantidad_de_valores_para_parametro = 3
2 cantidad_de_puntos_iniciales_por_parametro = 5
```

Ahora tendríamos dos parámetros a y b. Entonces deberíamos generar los rangos para esos valores.

Y como también tenemos dos variables, necesitamos las cotas inferiores para cada una de ellas.

```
1 cota_inferior_para_a = 0
2 cota_superior_para_a = 10
3
4 cota_inferior_para_b = 0
5 cota_superior_para_b = 10
6
7 cota_inferior_para_x0 = 0
8 cota_superior_para_x0 = 1000
9
10 cota_inferior_para_x1 = 0
11 cota_superior_para_x1 = 1000
```

Perfecto, además, vamos a crear ahora una lista para guardar los valores de los parámetros, y otra para los valores iniciales para cada combinación de parámetros.

Ahora mismo vamos a duplicar la información en el vector de los parámetros. Es decir vamos a tener varias veces los mismos valores de los parámetros, para que se vea como en la tablita que estaba más arriba.

Los nombres de los parámetros deberían ser: vector de parámetros aleatorios y vector de puntos iniciales

```

1 vector_de_parametros_aleatorios = []
2 vector_de_puntos_iniciales = []

```

Y ahora, vamos a generar los parámetros, solo que como son dos parámetros, necesitamos generar un a y un b. ¿Cómo juega eso con el parámetro de la cantidad de parámetros que se generan? :-/. Voy a asumir que lo que se generan la misma cantidad de muestras para cada parámetro.

```

1
2 # we start with the values for parameter a
3 for i in range (cantidad_de_valores_para_parametro):
4     # first we generate an initial value for the parameter
5     a = uniform(cota_inferior_para_a, cota_superior_para_a)
6     # once we have the value of a, generate the corresponding
7     # values for b.
8     for k in range (cantidad_de_valores_para_parametro):
9         b = uniform(cota_inferior_para_b, cota_superior_para_b)
10        # now the initial points
11        for j in range(cantidad_de_puntos_iniciales_por_parametro):
12            # first, we add the value of [a,b] to the vector of
13            ↪ parameters
14            vector_de_parametros_aleatorios.append([a, b])
15            # now we create the initial points
16            x0 = uniform(cota_inferior_para_x0,
17            ↪ cota_superior_para_x0)
18            x1 = uniform(cota_inferior_para_x1,
19            ↪ cota_superior_para_x1)
20            # and then we add it to the vector
21            vector_de_puntos_iniciales.append([x0, x1])

```

vamos a comprobar que los vectores se crearon como queríamos

```

1 for i in range(len(vector_de_parametros_aleatorios)):
2     print(vector_de_parametros_aleatorios[i],
3     ↪ vector_de_puntos_iniciales[i])

```

```

[7.967892272819678, 1.0982447039260324] [147.00895068857167, 375.15172455382384]
[7.967892272819678, 1.0982447039260324] [829.7188087836372, 989.1448956767717]
[7.967892272819678, 1.0982447039260324] [80.92569143629846, 37.49525086899064]
[7.967892272819678, 1.0982447039260324] [936.6903677905967, 607.7640697537013]

```

[7.967892272819678, 1.0982447039260324] [969.6641773285313, 230.81611400461745]
 [7.967892272819678, 8.316464983798488] [689.9823889387617, 645.4063757963818]
 [7.967892272819678, 8.316464983798488] [793.914928590146, 576.2808092616717]
 [7.967892272819678, 8.316464983798488] [804.3872176983352, 321.8633622594018]
 [7.967892272819678, 8.316464983798488] [814.5461465201743, 338.05089063658454]
 [7.967892272819678, 8.316464983798488] [342.55120056135604, 321.6615882680881]
 [7.967892272819678, 4.91240664550771] [0.9729096027673023, 193.5603192224128]
 [7.967892272819678, 4.91240664550771] [320.1698798123456, 532.1996543763946]
 [7.967892272819678, 4.91240664550771] [453.32421117522415, 253.3032980665565]
 [7.967892272819678, 4.91240664550771] [916.9905610484702, 160.12790461286065]
 [7.967892272819678, 4.91240664550771] [108.30656530024297, 701.4708224371416]
 [3.537421902354554, 6.257081507455001] [856.4865350524348, 478.13699840410817]
 [3.537421902354554, 6.257081507455001] [435.81978808398003, 708.0876960995575]
 [3.537421902354554, 6.257081507455001] [8.088628293387302, 202.28387287453197]
 [3.537421902354554, 6.257081507455001] [28.583949005112075, 614.7746278560289]
 [3.537421902354554, 6.257081507455001] [330.4167398090806, 948.5796404597589]
 [3.537421902354554, 0.6935440294356743] [18.541069931644614, 223.26636865989192]
 [3.537421902354554, 0.6935440294356743] [852.579415110743, 323.90982617976647]
 [3.537421902354554, 0.6935440294356743] [750.4659819317129, 968.1337551667722]
 [3.537421902354554, 0.6935440294356743] [690.7489030804229, 247.59790649773805]
 [3.537421902354554, 0.6935440294356743] [911.2801331603022, 789.9111857310149]
 [3.537421902354554, 5.942600029314777] [928.3598208586942, 564.9062722376917]
 [3.537421902354554, 5.942600029314777] [724.7881946670833, 283.4082170175595]
 [3.537421902354554, 5.942600029314777] [131.25531981915472, 107.55045765171467]
 [3.537421902354554, 5.942600029314777] [499.0595206900122, 267.16450966419234]
 [3.537421902354554, 5.942600029314777] [220.13017948893355, 34.98726051479262]
 [4.966152896336206, 4.740787251304943] [315.5781885286415, 924.4589508966142]
 [4.966152896336206, 4.740787251304943] [346.80692638554257, 603.5872159280515]
 [4.966152896336206, 4.740787251304943] [661.8718489377653, 937.3811966043751]
 [4.966152896336206, 4.740787251304943] [238.06540927654595, 366.9817415777501]
 [4.966152896336206, 4.740787251304943] [562.0440774439732, 147.15812032166608]
 [4.966152896336206, 2.4905776952077305] [852.5924615711035, 642.3966846413406]
 [4.966152896336206, 2.4905776952077305] [756.9032966841199, 760.6477272626395]
 [4.966152896336206, 2.4905776952077305] [96.7800372600336, 560.6901208366116]
 [4.966152896336206, 2.4905776952077305] [138.0985770185591, 621.6701147754444]
 [4.966152896336206, 2.4905776952077305] [251.05914868715251, 879.5208177287553]
 [4.966152896336206, 0.699712500581362] [723.3366986954632, 893.2043730324636]
 [4.966152896336206, 0.699712500581362] [853.6111141561192, 687.0634251378964]
 [4.966152896336206, 0.699712500581362] [842.2739604910943, 933.8741794340164]
 [4.966152896336206, 0.699712500581362] [956.103695548754, 551.0060076645764]

[4.966152896336206, 0.699712500581362] [54.02756308057721, 99.21168529900837]

Perfecto... ya tenemos los datos en la forma en que los queremos. Ahora vamos a crear el csv... :-/... pues no... primero vamos a encontrar los ceros para los juegos de datos que tenemos y lo vamos a almacenar en un otro vector.

Este otro vector tendrá dos elementos, el cero encontrado por el algoritmo y un valor booleano que indicará si se encontró un cero o no.

De hecho, vamos a hacer una función que reciba la función, los dos vectores (el de parámetros y el de los puntos iniciales), y devuelva un tercer vector (con la misma cantidad de elementos que el de parámetros) y que tenga el cero encontrado, el valor de la función y si fue un cero o no.

```
1 def find_zeros(f, params_vector, initial_values_vector):
2     found_solutions = []
3     print("iter", "success", "parameters", "solution", "function"
4           ↪ "value")
5     for i in range(len(params_vector)):
6         current_solution = root(f, initial_values_vector[i],
7                                 ↪ params_vector[i])
8         # let's store the data
9         found_solutions.append([current_solution.x,
10                                ↪ current_solution.fun, current_solution.success])
11         print(i+1, current_solution.success, params_vector[i],
12               ↪ current_solution.x, current_solution.fun)
13     return found_solutions
```

Vamos a comprobar si la función funciona:

```
1 found_zeros1 = find_zeros(function1,
2   ↪ vector_de_parametros_aleatorios, vector_de_puntos_iniciales)
3
4 # the following line is to print the results
5 # print(found_zeros1)
```

iter success parameters solution function value

```
1 True [7.967892272819678, 1.0982447039260324] [7.96789227 1.20614143] [-
1.34018202e-13 -1.35003120e-13]
2 True [7.967892272819678, 1.0982447039260324] [1.0982447 8.75069549] [-
8.54206181e-14 -8.70414851e-14]
```

3 True [7.967892272819678, 1.0982447039260324] [1.0982447 8.75069549] [-
 0.00000000e+00 -5.32907052e-15]
 4 True [7.967892272819678, 1.0982447039260324] [1.0982447 8.75069549] [6.34553163e-
 13 6.28830321e-13]
 5 True [7.967892272819678, 1.0982447039260324] [1.0982447 8.75069549] [1.09826509e-
 13 1.10134124e-13]
 6 True [7.967892272819678, 8.316464983798488] [7.96789227 69.16358983] [8.18765405e-
 13 7.95807864e-13]
 7 True [7.967892272819678, 8.316464983798488] [7.96789227 69.16358983] [2.78071270e-
 13 3.41060513e-13]
 8 True [7.967892272819678, 8.316464983798488] [7.96789227 69.16358983] [-
 1.04276726e-12 -9.09494702e-13]
 9 True [7.967892272819678, 8.316464983798488] [7.96789227 69.16358983] [-
 4.18909218e-12 -4.09272616e-12]
 10 True [7.967892272819678, 8.316464983798488] [7.96789227 69.16358983] [-
 6.74580302e-12 -6.70752343e-12]
 11 True [7.967892272819678, 4.91240664550771] [4.91240665 39.14152695] [-
 0. 0.]
 12 True [7.967892272819678, 4.91240664550771] [4.91240665 39.14152695] [-
 4.99342217e-13 -5.40012479e-13]
 13 True [7.967892272819678, 4.91240664550771] [4.91240665 39.14152695] [-
 6.51315935e-14 -5.68434189e-14]
 14 True [7.967892272819678, 4.91240664550771] [4.91240665 39.14152695] [-
 4.34210623e-14 -5.68434189e-14]
 15 True [7.967892272819678, 4.91240664550771] [4.91240665 39.14152695] [-
 6.05723820e-12 -6.05382411e-12]
 16 True [3.537421902354554, 6.257081507455001] [3.5374219 39.15106899] [3.77856231e-
 13 3.69482223e-13]
 17 True [3.537421902354554, 6.257081507455001] [3.5374219 39.15106899] [3.17399234e-
 13 3.41060513e-13]
 18 True [3.537421902354554, 6.257081507455001] [6.25708151 22.13393717] [-
 9.66217188e-15 -2.84217094e-14]
 19 True [3.537421902354554, 6.257081507455001] [6.25708151 22.13393717] [-
 1.35270406e-13 -1.42108547e-13]
 20 True [3.537421902354554, 6.257081507455001] [3.5374219 39.15106899] [6.80141216e-
 14 5.68434189e-14]
 21 True [3.537421902354554, 0.6935440294356743] [3.5374219 0.48100332] [-
 0.00000000e+00 4.4408921e-16]
 22 True [3.537421902354554, 0.6935440294356743] [0.69354403 2.45335784] [4.16768708e-
 14 4.17443857e-14]

23 True [3.537421902354554, 0.6935440294356743] [3.5374219 0.48100332] [-
 1.13867177e-14 -1.15463195e-14]
 24 True [3.537421902354554, 0.6935440294356743] [0.69354403 2.45335784] [-
 1.42711709e-13 -1.41886503e-13]
 25 True [3.537421902354554, 0.6935440294356743] [0.69354403 2.45335784] [4.16768708e-
 14 4.19664303e-14]
 26 True [3.537421902354554, 5.942600029314777] [3.5374219 35.31449511] [-
 5.77610873e-13 -5.68434189e-13]
 27 True [3.537421902354554, 5.942600029314777] [3.5374219 35.31449511] [-
 7.55337296e-13 -7.53175300e-13]
 28 True [3.537421902354554, 5.942600029314777] [3.5374219 35.31449511] [-
 6.34737223e-15 -1.42108547e-14]
 29 True [3.537421902354554, 5.942600029314777] [3.5374219 35.31449511] [-
 3.68147589e-13 -3.55271368e-13]
 30 True [3.537421902354554, 5.942600029314777] [3.5374219 35.31449511] [-
 3.51009684e-12 -3.51008111e-12]
 31 True [4.966152896336206, 4.740787251304943] [4.74078725 23.54347434] [-
 2.24184691e-14 -2.84217094e-14]
 32 True [4.966152896336206, 4.740787251304943] [4.74078725 23.54347434] [3.33875057e-
 13 3.12638804e-13]
 33 True [4.966152896336206, 4.740787251304943] [4.74078725 23.54347434] [3.45084292e-
 13 3.55271368e-13]
 34 True [4.966152896336206, 4.740787251304943] [4.74078725 23.54347434] [-
 3.60296824e-14 -1.42108547e-14]
 35 True [4.966152896336206, 4.740787251304943] [4.74078725 23.54347434] [-
 7.12587053e-14 -7.10542736e-14]
 36 True [4.966152896336206, 2.4905776952077305] [2.4905777 12.36858963] [-
 5.71675642e-14 -6.03961325e-14]
 37 True [4.966152896336206, 2.4905776952077305] [2.4905777 12.36858963] [-
 1.31925148e-13 -1.31450406e-13]
 38 True [4.966152896336206, 2.4905776952077305] [4.9661529 6.20297726] [-
 6.13330352e-13 -6.14619466e-13]
 39 True [4.966152896336206, 2.4905776952077305] [4.9661529 6.20297726] [-
 3.06665176e-13 -3.12638804e-13]
 40 True [4.966152896336206, 2.4905776952077305] [2.4905777 12.36858963] [-
 4.61738019e-13 -4.65405492e-13]
 41 True [4.966152896336206, 0.699712500581362] [4.9661529 0.48959758] [-
 5.30292553e-15 -5.32907052e-15]
 42 True [4.966152896336206, 0.699712500581362] [0.6997125 3.47487926] [1.62942492e-
 13 1.63424829e-13]

```

43 True [4.966152896336206, 0.699712500581362] [4.9661529  0.48959758] [-
5.70064494e-13 -5.68878278e-13]
44 True [4.966152896336206, 0.699712500581362] [4.9661529  0.48959758] [1.32573138e-
13 1.33226763e-13]
45 True [4.966152896336206, 0.699712500581362] [4.9661529  0.48959758] [1.85602393e-
14 1.73194792e-14]

```

Y ahora escribimos una función que cree el csv a partir de los tres vectores que le vamos a dar, el de los parámetros, el de los puntos iniciales y el de valores encontrados, si el algoritmo encontró algo.

El vector `found_zeros` tiene tres componentes:

posición 0 el cero encontrado por la función

posición 1 el valor de la función evaluada en el cero

posición 2 si el algoritmo considera que encontró un cero

Vamos a modificar el vector que finalmente exportaremos

```

1 def create_csv(params, found_zeros):
2     # the first thing is to create a new list
3     list_with_all_the_solutions = []
4     for i in range(len(params)):
5         # now we create a new list to "unpack" the arrays in
6         ↪ parameters and zeros
7         current_row = []
8         # if we found a zero (check position 2 of found_zeros)
9         if (found_zeros[2]):
10             # then we store the parameters and the zero found
11             # BUT, we are going to split the parameters, the
12             ↪ current_row
13             # does not contain any array.
14             # fist let's add all the parameters
15             for param in params[i]:
16                 current_row.append(param)
17             # now let's add the zeros found:
18             for zero in found_zeros[i][0]:
19                 current_row.append(zero)
20             #finally, we add the current_row to the
21             ↪ list_with_all_the_solutions
22             list_with_all_the_solutions.append(current_row)
23     # finally, we create the csv using pandas

```

```

21     df = pd.DataFrame(list_with_all_the_solutions)
22     df.to_csv('datosFR2.csv', index=False)
23     return list_with_all_the_solutions

```

Comprobemos que funciona:

```

1  l = create_csv(vector_de_parametros_aleatorios, found_zeros1)
2
3  for e in l:
4      print(e)

```

```

[7.967892272819678, 1.0982447039260324, 7.967892272819696, 1.2061414297015587]
[7.967892272819678, 1.0982447039260324, 1.0982447039260208, 8.750695490077367]
[7.967892272819678, 1.0982447039260324, 1.0982447039260317, 8.750695490077367]
[7.967892272819678, 1.0982447039260324, 1.0982447039261158, 8.750695490077275]
[7.967892272819678, 1.0982447039260324, 1.098244703926047, 8.750695490077351]
[7.967892272819678, 8.316464983798488, 7.96789227281996, 69.16358982674402]
[7.967892272819678, 8.316464983798488, 7.967892272819774, 69.16358982674558]
[7.967892272819678, 8.316464983798488, 7.967892272819318, 69.16358982674937]
[7.967892272819678, 8.316464983798488, 7.967892272818233, 69.1635898267584]
[7.967892272819678, 8.316464983798488, 7.967892272817351, 69.16358982676573]
[7.967892272819678, 4.91240664550771, 4.91240664550771, 39.14152695168892]
[7.967892272819678, 4.91240664550771, 4.912406645507675, 39.14152695168908]
[7.967892272819678, 4.91240664550771, 4.912406645507707, 39.14152695168894]
[7.967892272819678, 4.91240664550771, 4.912406645507708, 39.14152695168893]
[7.967892272819678, 4.91240664550771, 4.912406645507307, 39.1415269516909]
[3.537421902354554, 6.257081507455001, 3.537421902354576, 39.15106899093521]
[3.537421902354554, 6.257081507455001, 3.5374219023545725, 39.15106899093524]
[3.537421902354554, 6.257081507455001, 6.257081507455001, 22.133937169288966]
[3.537421902354554, 6.257081507455001, 6.257081507455009, 22.13393716928892]
[3.537421902354554, 6.257081507455001, 3.537421902354558, 39.15106899093532]
[3.537421902354554, 0.6935440294356743, 3.537421902354554, 0.4810033207658716]
[3.537421902354554, 0.6935440294356743, 0.6935440294356954, 2.453357839972971]
[3.537421902354554, 0.6935440294356743, 3.5374219023545597, 0.48100332076586744]
[3.537421902354554, 0.6935440294356743, 0.6935440294356022, 2.453357839973036]
[3.537421902354554, 0.6935440294356743, 0.6935440294356955, 2.453357839972971]
[3.537421902354554, 5.942600029314777, 3.5374219023545135, 35.31449510841223]
[3.537421902354554, 5.942600029314777, 3.537421902354501, 35.3144951084123]

```

[3.537421902354554, 5.942600029314777, 3.5374219023545534, 35.31449510841199]
 [3.537421902354554, 5.942600029314777, 3.537421902354528, 35.314495108412146]
 [3.537421902354554, 5.942600029314777, 3.5374219023543083, 35.314495108413446]
 [4.966152896336206, 4.740787251304943, 4.740787251304922, 23.543474338981905]
 [4.966152896336206, 4.740787251304943, 4.740787251305255, 23.543474338980324]
 [4.966152896336206, 4.740787251304943, 4.7407872513052665, 23.543474338980275]
 [4.966152896336206, 4.740787251304943, 4.74078725130491, 23.543474338981966]
 [4.966152896336206, 4.740787251304943, 4.7407872513048765, 23.543474338982122]
 [4.966152896336206, 2.4905776952077305, 2.490577695207721, 12.368589634606247]
 [4.966152896336206, 2.4905776952077305, 2.490577695207709, 12.368589634606277]
 [4.966152896336206, 2.4905776952077305, 4.966152896336306, 6.202977255866003]
 [4.966152896336206, 2.4905776952077305, 4.966152896336256, 6.202977255866126]
 [4.966152896336206, 2.4905776952077305, 2.4905776952076555, 12.36858963460641]
 [4.966152896336206, 0.699712500581362, 4.966152896336208, 0.4895975834698213]
 [4.966152896336206, 0.699712500581362, 0.6997125005814168, 3.4748792613647423]
 [4.966152896336206, 0.699712500581362, 4.966152896336397, 0.4895975834696892]
 [4.966152896336206, 0.699712500581362, 4.966152896336162, 0.48959758346985377]
 [4.966152896336206, 0.699712500581362, 4.9661528963362, 0.4895975834698266]

Y listo... con eso ya tenemos un csv que tiene, en las k primeras columnas los valores aleatorios de los k parámetros, y en las m últimas columnas, los valores de los ceros encontrados.

Supuestamente, eso es lo que debemos pasarle al algoritmo de regresión simbólica.

5 Transcripción del fichero s_symbolic_regressor.py

Al parecer, en este fichero se define un wrapper alrededor de la clase SymbolicRegressor, para poder cambiar la función objetivo sin demasiado trabajo. Supongo que eso es lo que haya hecho Guaty :-).

Por si acaso, lo voy a dejar todito como está ahí en un solo bloque de código.

```

1 from gplearn.genetic import SymbolicRegressor
2 from gplearn.fitness import make_fitness
3 import pickle
4 from pathlib import Path
5
6 class SymbolicRegressorWrapper(SymbolicRegressor):
7     def define_metric_function(self, fitness_fun,
8                               ↪ is_maximize=False):
```

```

8         fitness = make_fitness(function=fitness_fun,
9             ↪ greater_is_better=is_maximize)
10        self.metric = fitness
11
12    def set_new_target_generations(self, generations):
13        self.set_params(generations=generations, warm_start=True)
14
15    def save(self, name):
16        filename = f"models/{name}.pkl"
17        output_file = Path(filename)
18        output_file.parent.mkdir(exist_ok=True, parents=True)
19        with open(filename, "wb") as f:
20            pickle.dump(self, f)
21
22    @staticmethod
23    def load(name):
24        filename = f"models/{name}.pkl"
25        model = SymbolicRegressorWrapper()
26        with open(filename, "rb") as f:
27            model = pickle.load(f)
28        return model

```

6 transcripción del fichero Regression__update1.py

Lo primero es cargar lo que se está usando

```

1 import numpy as np
2 import graphviz
3 import pandas as pd

```

Esto es para no tener que hacerlo aquí también

```

1 from s_symbolic_regressor import SymbolicRegressorWrapper

```

Lo siguiente define la cantidad de parámetros que tiene el modelo.

Ahora mismo me gustaría que esa fuera la primera fila del csv: la cantidad de variables y la cantidad de parámetros... pero como ahora mismo lo único que estoy haciendo es transcribir el fichero, lo dejo aquí.

```
1 Parametros_del_modelos = 1
```

Lo siguiente es cargar el archivo CSV, que evidentemente, Pandas lo hace sin ningún tipo de problema.

```
1 archivo_csv = "datosFR2.csv"
2 df = pd.read_csv(archivo_csv, header=0)
```

Comprobemos que el `dataframe` tiene lo que se supone que debe tener:

```
1 print(df)
```

Resultado:

		0	1
2	0	4.431761	87.042034
3	1	4.431761	87.042034
4	2	4.431761	87.042034
5	3	4.431761	87.042034
6	4	4.431761	87.042034
7	5	3.997722	15.981782
8	6	3.997722	63.890724
9	7	3.997722	63.890724
10	8	3.997722	63.890724
11	9	3.997722	63.890724
12	10	9.511825	90.474817
13	11	9.511825	860.580636
14	12	9.511825	90.474817
15	13	9.511825	860.580636
16	14	9.511825	860.580636

Ahora vamos a separar lo que hay en el `dataframe` en dos listas, una con los valores de los parámetros, y otra con los valores de las soluciones.

```
1 parametros = df.iloc[:, :Parametros_del_modelos].values.tolist()
2 soluciones = df.iloc[:, Parametros_del_modelos:].values.tolist()
```

Comprobemos que en las listas tenemos los valores apropiados

```
1 print(soluciones)
```

Resultado:

```
1 [[87.0420341284018], [87.04203412840181],  
  ↪ [87.0420341284018], [87.04203412840181],  
  ↪ [87.04203412840181], [15.981782106235723],  
  ↪ [63.89072375816521], [63.8907237581641],  
  ↪ [63.89072375816402], [63.89072375816402],  
  ↪ [90.4748169294283], [860.5806355214845],  
  ↪ [90.47481692942826], [860.5806355214849],  
  ↪ [860.5806355214183]]
```

A continuación en el fichero `Regression_update1` hay un ciclo que no entiendo para qué es... quizás sea para quitar los ceros... no estoy muy seguro de cuál es la idea, pero está ahí. Aquí, en modo muy experimental, vamos a ignorarlo, y vamos directo a la función en la que definimos el fitness.

Vamos a convertir las liastas de parámetros y soluciones en arreglos de numpy:

```
1 parametros = np.array(parametros)  
2 soluciones = np.array(soluciones).T
```

Lo siguiente es solo para comprobar.

```
1 print(soluciones)
```

Resultado:

```
1 [[ 87.04203413  87.04203413  87.04203413  87.04203413  
  ↪  87.04203413  
2   15.98178211  63.89072376  63.89072376  63.89072376  
  ↪  63.89072376  
3   90.47481693  860.58063552  90.47481693  860.58063552  
  ↪  860.58063552]]
```

El próximo paso es definir la función de fitness. La idea es contar cuántas veces el candidato acierta con su predicción.

```
1 def define_fitness():
2     # error = 0.03 da exacto
3     error = 0.01 # "bastante bien" :D
4
5     def fitness(y, y_pred, w):
6         suma = 0
7
8         for i in range(len(y)):
9             if abs(y[i] - y_pred[i]) <= error:
10                 suma += 1
11         return suma
12
13     return fitness
```

Aquí deberíamos incorporar un ejemplo para comprobar que la función funciona como dice que funciona.

```
1 f0 = define_fitness()
2 xt0 = [1,2,3,4,5,6]
3 yt0 = [1,0,0,4,0,6]
4 print(f0(xt0, yt0, None))
```

Resultado:

```
1 3
```

El resultado anterior, solo comprueba que la función parece hacer lo que dice que hace.

Y ahora es cuando es... parece que va a invocar a la regresión simbólica.

```
1 modelos = []
2
3 cubic_gp = SymbolicRegressorWrapper(
4     population_size=100,
5     generations=30,
6     stopping_criteria=23,
```

```

7     p_crossover=0.7,
8     p_subtree_mutation=0.1,
9     p_hoist_mutation=0.05,
10    p_point_mutation=0.1,
11    max_samples=0.9,
12    verbose=1,
13    parsimony_coefficient=0.5,
14    random_state=0,
15 )
16
17 cubic_gp.define_metric_function(define_fitness(), is_maximize=True)

```

Y bueno, después de eso, me parece que ya lo que queda es invocar el método de la regresión simbólica.

```

1 i = 0
2
3 cubic_gp.fit(parametros, soluciones[i])

```

Population Average			Best Individual			
Gen	Length	Fitness	Length	Fitness	OOB Fitness	Time Left
0	45.88	0	63	0	0	3.00s
1	7.62	0.03	3	3	3	1.61s
2	2.94	0.66	5	12	12	1.32s
3	3.78	2.91	5	12	12	1.27s
4	4.74	5.67	5	12	12	1.26s
5	5.60	5.85	5	12	12	1.27s
6	6.64	6.54	5	12	12	1.24s
7	4.96	6.42	5	12	12	1.07s
8	5.24	5.88	5	12	12	1.03s
9	4.80	6.3	5	12	12	0.95s
10	5.42	5.94	5	12	12	0.92s
11	5.54	5.34	5	12	12	0.92s
12	7.40	5.4	5	12	12	0.94s
13	6.24	4.95	5	12	12	0.90s
14	5.52	5.4	5	12	12	0.78s
15	5.58	6.03	5	12	12	0.70s
16	5.00	5.34	5	12	12	0.65s

17	5.16	6.99	5	12	12	0.62s
18	5.40	6.57	5	12	12	0.54s
19	5.00	5.25	5	12	12	0.51s
20	4.74	5.4	5	12	12	0.45s
21	5.70	6.42	5	12	12	0.42s
22	4.78	6.18	5	12	12	0.35s
23	4.96	5.16	5	12	12	0.31s
24	5.20	5.97	5	12	12	0.25s
25	5.26	5.7	5	12	12	0.21s
26	5.18	5.85	5	12	12	0.15s
27	5.60	5.7	5	12	12	0.11s
28	4.96	5.61	5	12	12	0.05s
29	4.94	5.13	5	12	12	0.00s

Veamos qué devolvió el método de regresión simbólica:

```
1 print(cubic_gp)
```

Resultado:

```
1 mul(X0, mul(X0, X0))
```

Perfecto, esto es la función $y = x^3$, que es lo que estamos buscando :-).

Quizás podamos tener la tarea de convertirlo a algo más potable, como hacen en <https://medium.com/analytics-vidhya/python-symbolic-regression-with-gplearn-c> con el `converter`.

Lo primero es guardarlo en una lista de soluciones.

```
1 modelos.append(cubic_gp)
```

comprobemos ahora el valor de modelos:

```
1 for m in modelos:
2     print(m)
```

Resultado:

```
1 mul(X0, mul(X0, X0))
```

El próximo paso sería evaluar la función devuelta por `gplearn` (eso se hace con `predict`), para eliminar los datos que ya detectó.

Voy a tratar de hacer eso con el código. La idea es: por la nueva función objetivo que tenemos, hay varios elementos del vector de parámetros, que evaluados en `parámetros` obtiene el correspondiente valor de `soluciones`.

La idea es eliminar esos elementos.

Hagámoslo la primera vez, ahí más o menos como salga.

Veamos qué tenemos en `parámetros` y `soluciones`:

```
1 print(soluciones)
2 print(parametros)
```

Resultado:

```
1  [[ 87.04203413  87.04203413  87.04203413  87.04203413
   ↪  87.04203413
2    15.98178211  63.89072376  63.89072376  63.89072376
   ↪  63.89072376
3    90.47481693  860.58063552  90.47481693  860.58063552
   ↪  860.58063552]]
4  [[4.43176113]
5   [4.43176113]
6   [4.43176113]
7   [4.43176113]
8   [4.43176113]
9   [3.99772211]
10  [3.99772211]
11  [3.99772211]
12  [3.99772211]
13  [3.99772211]
14  [9.51182511]
15  [9.51182511]
16  [9.51182511]
17  [9.51182511]
18  [9.51182511]]
```

hagamos un ciclo por cada una:

```
1  for i in range(len(parametros)):
2      print (i, parametros[i], soluciones[0][i])
```

Resultado:

```
1 0 [4.43176113] 87.0420341284018
2 1 [4.43176113] 87.04203412840181
3 2 [4.43176113] 87.0420341284018
4 3 [4.43176113] 87.04203412840181
5 4 [4.43176113] 87.04203412840181
6 5 [3.99772211] 15.981782106235723
7 6 [3.99772211] 63.89072375816521
8 7 [3.99772211] 63.8907237581641
9 8 [3.99772211] 63.89072375816402
10 9 [3.99772211] 63.89072375816402
11 10 [9.51182511] 90.4748169294283
12 11 [9.51182511] 860.5806355214845
13 12 [9.51182511] 90.47481692942826
14 13 [9.51182511] 860.5806355214849
15 14 [9.51182511] 860.5806355214183
```

NOTA: Hay que revisar aquí por qué soluciones está indizado en 0.

En cualquier caso, el próximo paso es eliminar de estas dos listas los elementos que `cubic_gp` interpola, para el error que definimos.

Allá va eso.

Nota: comparamos con el módulo de la diferencia menor que una tolerancia porque cuando estamos trabajando con números reales, no se debe usar el operador `==`.

```
1 new_parameters = []
2 new_solutions = []
3
4 for i in range(len(parametros)):
5     if (abs(cubic_gp.predict(parametros[i].reshape(1,-1))) -
6         ↪ soluciones[0][i]) < 0.03:
7         # no hacemos nada
8         continue
9     else:
10        # agregamos a la lista nueva
11        new_parameters.append(parametros[i])
12        new_solutions.append(soluciones[0][i])
```

Vamos a ver qué tenemos en las nuevas listas:

```
1 print(new_parameters)
2 print(new_solutions)
```

Resultado:

```
1 [array([3.99772211]), array([9.51182511]),
   ↪ array([9.51182511])]
2 [15.981782106235723, 90.4748169294283, 90.47481692942826]
```

Y bueno... supuestamente, la gracia es que ahora volvemos a llamar a la regresión simbólica.

Voy a crear otra instancia, pero le voy a pasar parámetros más pequeños.

```
1 cubic_gp = SymbolicRegressorWrapper(
2     population_size=100,
3     generations=30,
4     stopping_criteria=23,
5     p_crossover=0.7,
6     p_subtree_mutation=0.1,
7     p_hoist_mutation=0.05,
8     p_point_mutation=0.1,
9     max_samples=0.9,
10    verbose=1,
11    parsimony_coefficient=0.5,
12    random_state=0,
13 )
14
15 cubic_gp.define_metric_function(define_fitness(), is_maximize=True)
```

Y ahora le hacemos otra vez el fit.

```
1 cubic_gp.fit(new_parameters, new_solutions)
```

Resultado:

1	Population Average		Best Individual		
2	↪				
3	Gen	Length	Fitness	Length	Fitness
4	↪ 0	00B Fitness	Time Left		
5	1	45.88	0	63	0
6	↪ 0	3.08s			
7	1	7.62	0.03	3	3
8	↪ 3	1.63s			
9	2	2.94	0.54	3	3
10	↪ 3	1.30s			
11	3	3.54	1.98	3	3
12	↪ 3	1.26s			
13	4	3.32	2.22	3	3
14	↪ 3	1.22s			
15	5	3.80	1.98	3	3
16	↪ 3	1.25s			
17	6	4.74	2.16	3	3
18	↪ 3	1.20s			
19	7	2.86	2.1	3	3
20	↪ 3	1.03s			
21	8	3.18	2.13	3	3
22	↪ 3	0.99s			
23	9	3.08	2.19	3	3
24	↪ 3	0.92s			
25	10	3.26	2.13	3	3
26	↪ 3	0.88s			
27	11	3.48	1.92	3	3
28	↪ 3	0.96s			
29	12	5.30	2.04	3	3
30	↪ 3	0.89s			
31	13	4.54	1.89	3	3
32	↪ 3	0.87s			
33	14	3.74	1.77	3	3
34	↪ 3	0.76s			
35	15	3.94	2.22	3	3
36	↪ 3	0.68s			
37	16	3.10	2.04	3	3
38	↪ 3	0.63s			
39	17	3.38	1.95	3	3
40	↪ 3	0.60s			
41	18	3.54	2.19	3	3
42	↪ 3	0.54s			
43	19	3.08	2.042	3	3
44	↪ 3	0.49s			
45	20	2.92	1.98	3	3
46	↪ 3	0.44s			
47	21	3.80	1.83	3	3
48	↪ 3	0.41s			
49	22	3.14	2.31	3	3
50	↪ 3	0.34s			
51	23	3.18	2.01	3	3

Veamos qué fue lo que obtuvimos:

```
1 print(cubic_gp)
```

Resultado:

```
1 mul(X0, X0)
```

Perfecto... y eso es justamente la función cuadrática.

Por si acaso, dice el “proctolo”, que lo primero es agregarlo a la lista de modelos:

```
1 modelos.append(cubic_gp)
```

Veamos qué tenemos en modelos:

```
1 for m in modelos:
2     print (m)
```

Resultado:

```
1 mul(X0, mul(X0, X0))
2 mul(X0, X0)
```

Por si acaso, vamos a realizar otra vez el proceso de iterar por los parámetros y las soluciones eliminando las que se “interpolan”.

```
1 new_parameters2 = []
2 new_solutions2 = []
3
4 for i in range(len(new_parameters2)):
5     if (abs(cubic_gp.predict(new_parameters2[i].reshape(1,-1))) -
        ↪ new_solutions2[0][i]) < 0.03:
6         # no hacemos nada
7         continue
8     else:
```

```
9      # agregamos a la lista nueva  
10     new_parameters2.append(parametros[i])  
11     new_solutions2.append(soluciones[0][i])
```

Vamos a ver qué tenemos en las nuevas listas:

```
1 print(new_parameters2)  
2 print(new_solutions2)
```

Resultado:

```
1 []  
2 []
```

Y como ahora las dos listas están vacías, supuestamente lo que tenemos en `modelos` es lo que estamos buscando.